INTRODUCTION

What Is Image Traversal — and Why It Matters

Definition

An image is stored as a 2D array of pixel values. Image traversal is the systematic order in which these pixel locations are visited by an algorithm.

Traversal order affects:

- Computational efficiency (cache locality, memory access pattern)
- Compression performance (grouping similar values together)
- Correctness of region-based operations (segmentation)
- Suitability for hierarchical / multi-resolution analysis

Classification of Traversal Methods

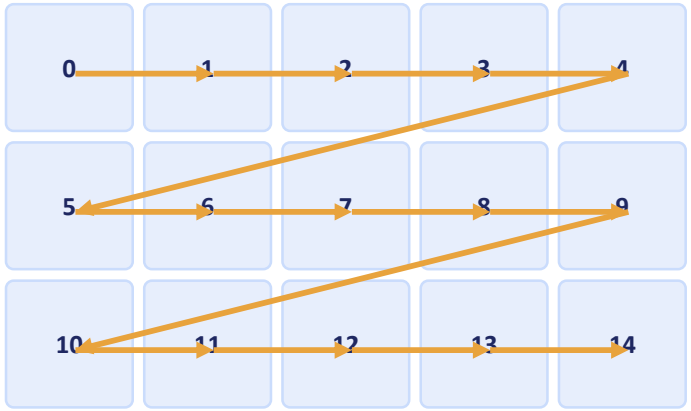
- **Linear / Sequential**
  Raster · Boustrophedon · Zigzag (JPEG)
- **Geometric Pattern**
  Spiral Traversal
- **Space-Filling Curves**
  Z-order (Morton) · Hilbert Curve
- **Hierarchical**
  Quadtree Traversal
- **Region / Structure-Based**
  Flood Fill · Skeleton · Contour Tracing

Through the lens of Optimization Criteria and Objective Functions.
A matrix is just a raw two-dimensional vector space. When we serialize (flatten) it into a 1D memory array, we are forced to choose an access path. Every method fills the 1D array differently because each algorithm is optimizing for a completely different engineering bottleneck.

Row-Major Order (The Baseline)
The Path: Straight horizontal lines, row by row (0, 1, 2, 3, then 4, 5, 6, 7).
What it Optimizes: Sequential 1D Array Access.
The Logic: This is how programming languages like C, C++, and Python (NumPy) native store arrays in RAM. It assumes you are going to process the matrix linearly from the very first element to the absolute last element.

JPEG Zigzag Traversal
The Path: Sweeping, alternating anti-diagonals.
What it Optimizes: Spatial Frequency and Entropy Coding.
The Logic: After a Discrete Cosine Transform (DCT) and Quantization, a matrix no longer represents spatial pixels—it represents frequencies. The top-left corner holds low frequencies (smooth gradients/shapes), while the bottom-right holds high frequencies (sharp noise/edges).

Raster Scan (Row-Major) Traversal



Definition

Pixels visited row by row, left to right, top to bottom — matching the natural memory layout of a 2D array (row-major order).

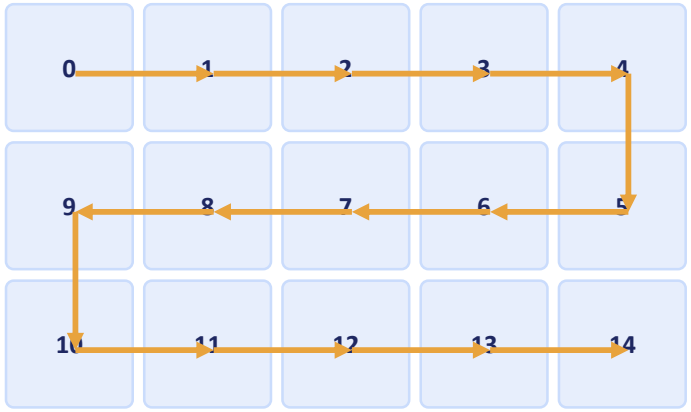
Algorithm

```
for row = 0 to height-1:
    for col = 0 to width-1:
        visit(image[row][col])
```

Properties & Applications

- Complexity O(H×W); excellent cache locality within a row
- Discontinuity between end of one row and start of next
- Used in: display rendering, basic filters, video scanning

Boustrophedon (Zigzag Row) Traversal



Definition

Named after the ox-plowing pattern — alternates scan direction each row, avoiding the large jump of raster scan.

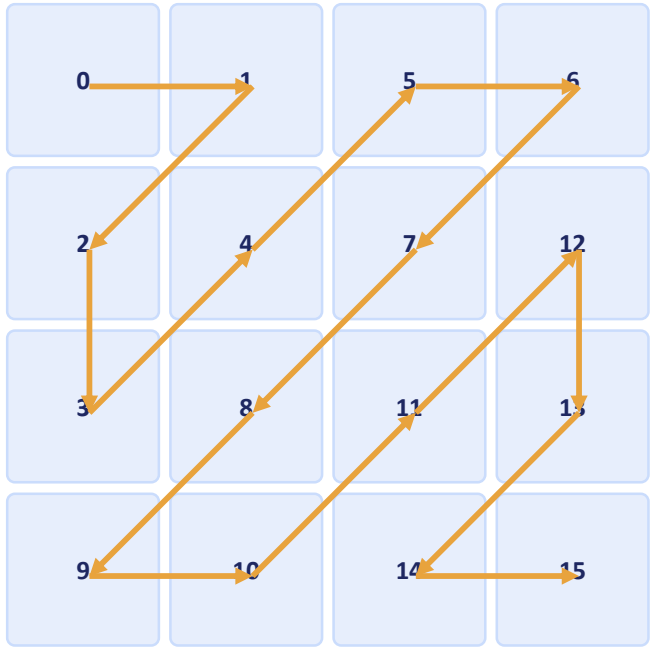
Algorithm

```
for row = 0 to height-1:
  if row is even: scan left → right
  else: scan right → left
```

Properties & Applications

- Maintains spatial continuity between consecutive rows
- Slightly more complex indexing than raster scan
- Used in: hardware scanners, dot-matrix/thermal printers, robotic scan paths

Zigzag Diagonal Traversal (JPEG-Style)



fundamentally different — it scans frequency-domain coefficients, not spatial pixel intensities.

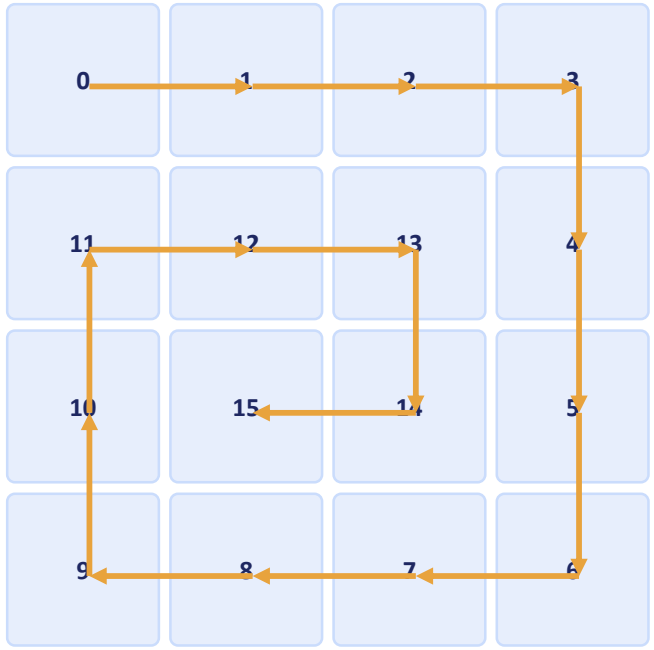
Why Diagonal?

Applied to DCT coefficient blocks, not raw pixels. After the DCT and quantization, energy concentrates at the top-left (low frequency); the diagonal zigzag groups the trailing zeros together for efficient Run-Length Encoding.

Applications

- JPEG still-image compression
- MPEG intra-frame video coding

Spiral Traversal



Algorithm (outside-in)

```
top=0, bottom=H-1, left=0, right=W-1
while top<=bottom and left<=right:
  scan row 'top' →, col 'right' ↓,
  row 'bottom' ←, col 'left' ↑;
  shrink each boundary
```

Properties & Applications

- Emphasizes the center (inside-out) or periphery (outside-in)
- More complex boundary bookkeeping than raster scan
- Used in: radial/center-weighted analysis, iris & pupil detection, texture synthesis

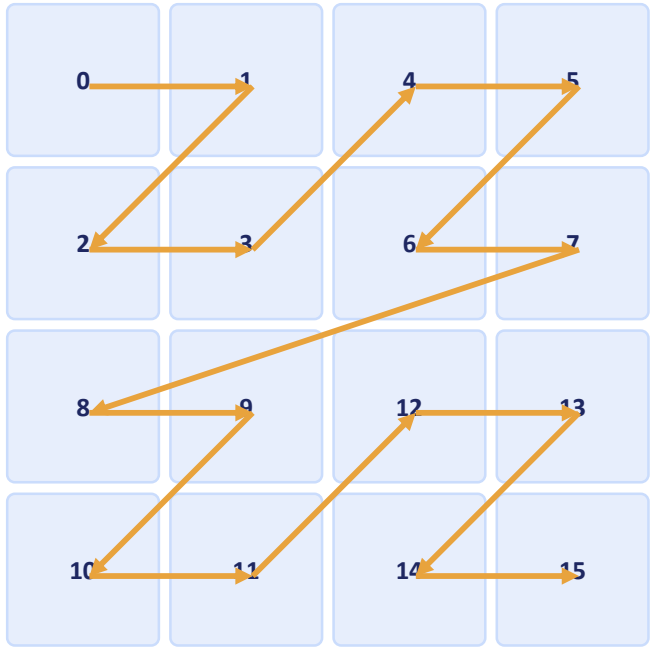
Z-Order Traversal (Morton Coding)

The Path: A recursive, fractal, bit-interleaved "Z" shape.

What it Optimizes: Multi-directional Spatial Locality (Cache Tuning).

The Logic: Unlike Row-Major (which only cares about horizontal neighbors), Z-ordering treats horizontal and vertical proximity with equal weight. It forces clusters of pixels (2×2, then 4×4 blocks) to sit inside contiguous memory addresses.

Z-Order (Morton Order) Traversal



Space-Filling Curves. Z-Order Traversal (Morton Order) maps N-dimensional space into 1D space while preserving local neighborhoods to minimize cache misses.

Bit-Interleaving Example

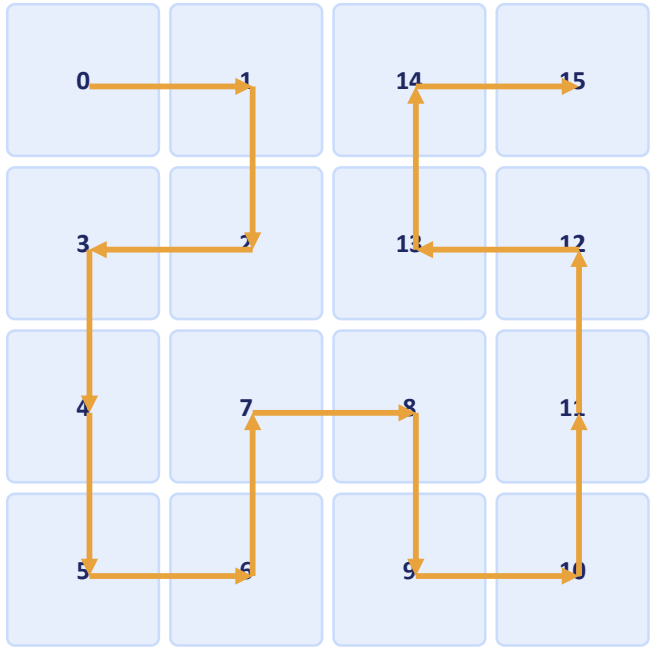
```
y = 2 = 10    x = 3 = 11
Interleave (y,x,y,x): 1 1 0 1 = 13
```

Properties & Applications

- Interleaves bits of x, y coordinates into one index
- Fast to compute (hardware-friendly); mostly preserves locality
- Occasional large jumps at quadrant boundaries
- Used in: quadtree indexing, spatial databases, GPU texture layout

SPACE-FILLING CURVE · 02

Hilbert Curve Traversal



Key Property

No discontinuous jumps: consecutive points on the Hilbert curve (1D index) are always spatially adjacent in 2D — the strongest locality guarantee among common space-filling curves.

Applications

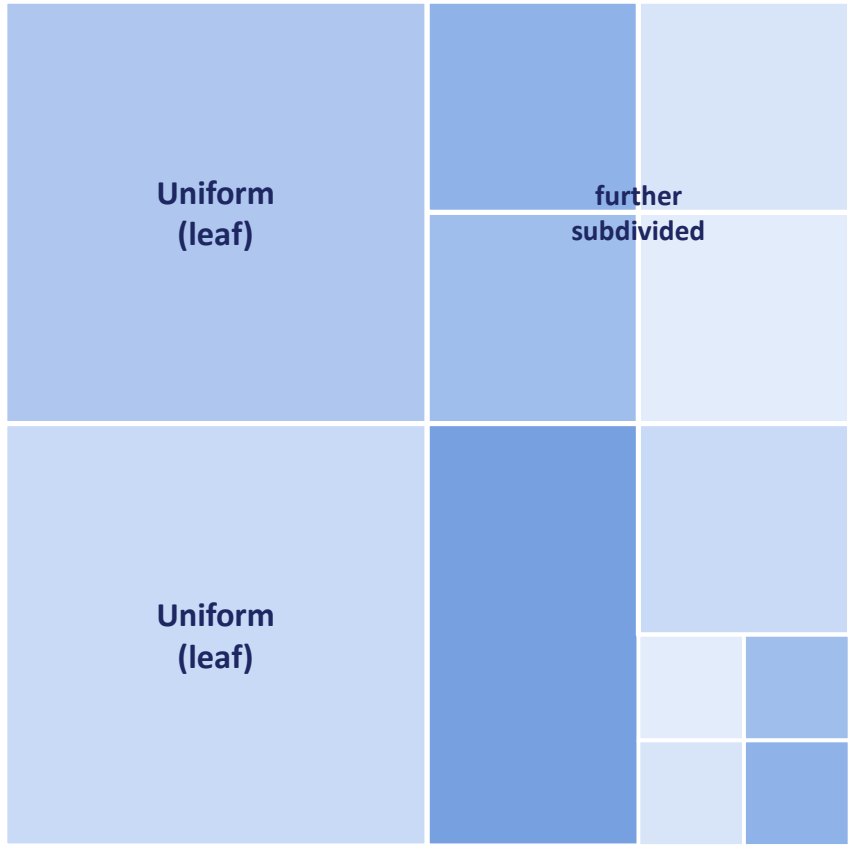
- Image / video compression (locality-based prediction)
- Caching and memory-hierarchy optimization
- GIS and satellite image indexing
- Clustering & nearest-neighbor search in image databases

COMPARISON

Z-Order vs. Hilbert Curve

Criterion	Z-Order (Morton)	Hilbert Curve
Computation speed	Faster (bitwise ops)	Slower (recursive)
Locality preservation	Good, with occasional jumps	Excellent, no jumps
Implementation complexity	Simple	Moderate to complex
Common use	GPU memory, quadtrees	Compression, GIS, caching

Quadtree Traversal



Recursive subdivision — stops where a region is homogeneous

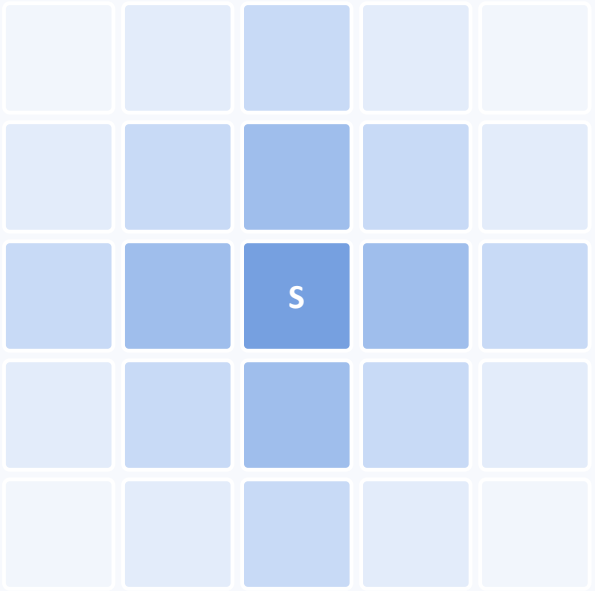
Algorithm

```
function buildQuadtree(region):
  if region is homogeneous or 1 pixel:
    mark as leaf, store value
  else:
    split into TL, TR, BL, BR
    recurse on each quadrant
```

Properties & Applications

- Traversal order is content-driven, not fixed geometry
- Efficient for images with large uniform regions
- Overhead grows for highly detailed / textured images
- Used in: compression, region segmentation, adaptive mesh generation, LOD rendering

Flood Fill Traversal (BFS / DFS)



BFS growth: uniform layer-by-layer expansion from seed (S)

Connectivity & Algorithms

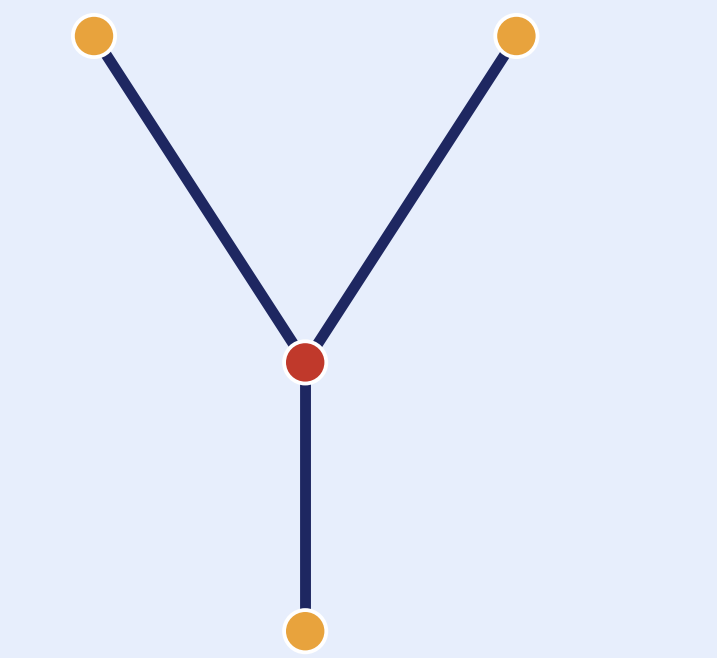
4-connectivity: up/down/left/right · 8-connectivity: + diagonals

```
BFS (queue): expand all neighbors at current distance before going
further
DFS (stack/recursion): explore deep along one path, then backtrack
```

Aspect	BFS	DFS
Data structure	Queue	Stack / recursion
Memory risk	High on wide regions	Stack limit on deep regions

Applications: paint-bucket tool, region growing segmentation, connected-component labeling, blob counting

Skeleton (Morphological) Traversal



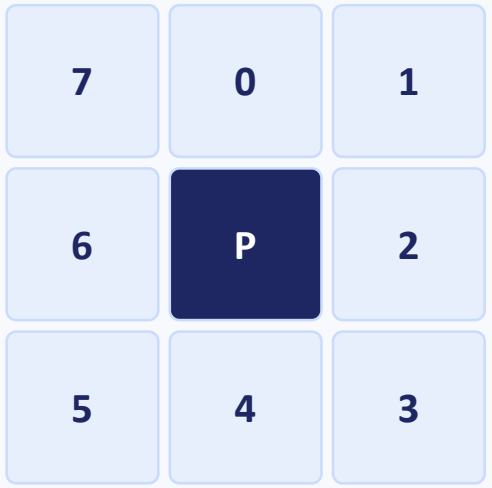
Process

1. Skeletonize (thin) the binary region
2. Traverse skeleton via 8-neighborhood
3. Track endpoints, branch points, path segments

Applications & Limitations

- OCR — character stroke/topology analysis
- Fingerprint ridge tracing and minutiae extraction
- Vascular / road network extraction
- Limitation: sensitive to noise — spurious short branches

Contour Traversal (Moore-Neighbor Tracing)



8-neighborhood examined clockwise from the arrival direction

Algorithm

1. Find a starting boundary pixel
2. Examine 8 neighbors in fixed order, starting just after the arrival direction
3. First foreground pixel = next boundary point
4. Repeat until back at the start

Applications & Limitations

- OpenCV's findContours() function
- Shape descriptors — perimeter, chain codes, Fourier descriptors
- Limitation: can fail on single-pixel-wide connections

SUMMARY

Comparative Overview of All Methods

Method	Pattern Type	Best Suited For
Raster Scan	Linear, row-major	General processing
Boustrophedon	Linear, alternating	Hardware scanning
Zigzag (JPEG)	Diagonal (freq. domain)	Compression (DCT coeffs)
Spiral	Rectangular spiral	Center-focused analysis
Z-order (Morton)	Recursive quadrant	Spatial indexing, GPU
Hilbert Curve	Recursive, locality-optimal	Compression, caching, GIS
Quadtree	Hierarchical, adaptive	Segmentation, compression
Flood Fill (BFS/DFS)	Region-based	Connected components
Skeleton Traversal	Topological	OCR, ridge tracing
Contour Tracing	Boundary-following	Shape analysis